

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengantar Mikrokontroler & Dasar Pemrograman

Athaya Ahmad Farrel Hatrian - 5024241068

2026

1 Langkah-Langkah Percobaan

Langkah-langkah percobaan pada Modul 1 dilakukan melalui lima tahapan utama dengan menggunakan board STM32 Blackpill dan ESP32 DevKit. Setiap percobaan dilakukan secara berurutan sesuai dengan instruksi pada modul. Adapun langkah-langkah yang dilakukan adalah sebagai berikut.

1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Menyiapkan board STM32 Blackpill, ST-Link V2, kabel jumper female-female, serta laptop atau PC yang telah terpasang Visual Studio Code dan ekstensi PlatformIO.
2. Membuka project PlatformIO Modul 1 yang telah dibuat sebelumnya.
3. Mengatur konfigurasi pada file `platformio.ini` dengan memilih platform STM32, board `blackpill_f401cc`, framework Arduino, dan upload protocol ST-Link.
4. Menghubungkan ST-Link dengan board STM32 Blackpill menggunakan kabel jumper. Pin SWDIO dihubungkan ke SWDIO, SWCLK ke SWCLK, GND ke GND, dan 3.3V ke 3V3.
5. Memastikan ST-Link menggunakan tegangan 3.3V sebelum dihubungkan ke komputer melalui kabel USB.
6. Membuka file `src/main.cpp`, kemudian menuliskan program Blink menggunakan pin PC13 sebagai output LED onboard.
7. Melakukan proses Build untuk memeriksa apakah program dapat dikompilasi tanpa kesalahan.
8. Setelah proses Build berhasil, melakukan Upload untuk mengunggah program ke board STM32 melalui ST-Link.
9. Mengamati LED onboard pada pin PC13. LED akan menyala selama 500 ms dan mati selama 500 ms secara berulang.

1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Menyiapkan board ESP32 DevKit dan kabel USB untuk menghubungkan board dengan komputer.
2. Membuka kembali project PlatformIO Modul 1 yang sama dengan percobaan sebelumnya.
3. Mengubah konfigurasi pada file `platformio.ini` dengan memilih platform `espressif32`, board `esp32dev`, dan framework `espidf`.
4. Menunggu proses refresh atau re-index konfigurasi PlatformIO hingga selesai sebelum melanjutkan proses pemrograman.
5. Menghubungkan ESP32 ke komputer menggunakan kabel USB dan memastikan komputer dapat mengenali port komunikasi board.
6. Menghapus file `src/main.cpp`, kemudian membuat file baru bernama `src/main.c` karena program menggunakan framework ESP-IDF.

7. Menuliskan program Blink dengan menggunakan fungsi GPIO dari ESP-IDF. Pin GPIO 2 digunakan sebagai output untuk mengendalikan LED onboard.
8. Melakukan proses Build untuk memastikan program berhasil dikompilasi.
9. Melakukan Upload untuk mengunggah program ke board ESP32.
10. Mengamati LED onboard ESP32 yang menyala dan mati secara bergantian dengan interval 500 ms.
11. Mengamati struktur program dan fungsi yang digunakan, kemudian membandingkannya dengan program Blink pada percobaan pertama yang menggunakan framework Arduino.

1.3 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal

1. Menyiapkan board ESP32 DevKit, breadboard, dua pushbutton, tiga resistor 10kΩ, kabel jumper, dan kabel USB.
2. Mengubah konfigurasi `platformio.ini` menggunakan platform `espressif32`, board `esp32dev`, dan framework Arduino.
3. Menghapus file `src/main.c`, kemudian membuat kembali file `src/main.cpp` untuk menyesuaikan penggunaan framework Arduino.
4. Memasang dua pushbutton pada breadboard dan menghubungkannya dengan ESP32 sesuai skema rangkaian pull-up dan pull-down eksternal.
5. Menghubungkan pushbutton pertama ke GPIO 32 dengan konfigurasi pull-up eksternal. Salah satu kaki tombol dihubungkan ke GPIO 32 dan 3.3V melalui resistor 10kΩ, sedangkan kaki lainnya dihubungkan ke GND.
6. Menghubungkan pushbutton kedua ke GPIO 33 dengan konfigurasi pull-down eksternal. Salah satu kaki tombol dihubungkan ke GPIO 33 dan GND melalui resistor 10kΩ, sedangkan kaki lainnya dihubungkan ke 3.3V.
7. Memastikan seluruh sambungan rangkaian telah sesuai sebelum menghubungkan ESP32 ke komputer.
8. Menuliskan Kode Program A pada file `src/main.cpp`. Program menggunakan konfigurasi INPUT untuk membaca kondisi kedua tombol karena resistor pull-up dan pull-down telah dipasang secara eksternal.
9. Melakukan Build dan Upload program ke ESP32.
10. Membuka Serial Monitor dengan baud rate 115200 untuk mengamati kondisi kedua pushbutton.
11. Menekan dan melepaskan masing-masing tombol, kemudian mencatat perubahan kondisi yang ditampilkan pada Serial Monitor.
12. Setelah pengujian eksternal selesai, melepas resistor pull-up dan pull-down dari rangkaian tanpa mengubah posisi kedua pushbutton.

13. Mengatur kembali rangkaian untuk menggunakan pull-up dan pull-down internal. Pushbutton pertama dihubungkan antara GPIO 32 dan GND, sedangkan pushbutton kedua dihubungkan antara GPIO 33 dan 3.3V.

14. Mengubah program menggunakan Kode Program B dengan konfigurasi INPUT_PULLUP pada GPIO 32 dan INPUT_PULLDOWN pada GPIO 33.

15. Melakukan Build dan Upload kembali, kemudian membuka Serial Monitor untuk mengamati hasil pembacaan tombol menggunakan resistor internal.

16. Membandingkan hasil pengujian pull-up dan pull-down eksternal dengan internal. Hasil yang diharapkan adalah kedua metode menghasilkan kondisi logika yang sama, yaitu HIGH saat tombol pull-up tidak ditekan dan LOW saat ditekan, sedangkan pull-down menghasilkan kondisi sebaliknya.

1.4 Percobaan 4: Debouncing pada Input Tombol GPIO

1. Menyiapkan board ESP32 DevKit, satu pushbutton, breadboard, kabel jumper, dan kabel USB.

2. Memastikan konfigurasi `platformio.ini` masih menggunakan board `esp32dev` dengan framework Arduino seperti pada percobaan sebelumnya.

3. Memasang pushbutton pada breadboard dengan menghubungkan salah satu kaki tombol ke GPIO 25 dan kaki lainnya ke GND.

4. Mengatur pin GPIO 25 menggunakan konfigurasi pull-up internal sehingga kondisi awal tombol berada pada logika HIGH.

5. Membuka file `src/main.cpp`, kemudian menuliskan program debouncing yang memiliki dua penghitung, yaitu `counterNoDebounce` dan `counterWithDebounce`.

6. Mengatur nilai awal `DEBOUNCE_DELAY` sebesar 50 ms sebagai waktu penyaringan perubahan sinyal tombol.

7. Melakukan Build dan Upload program ke ESP32.

8. Membuka Serial Monitor dengan baud rate 115200 untuk mengamati hasil kedua penghitung yang ditampilkan secara bersamaan.

9. Menekan tombol satu kali dan membandingkan perubahan nilai penghitung tanpa debouncing dengan penghitung yang menggunakan debouncing.

10. Mengulangi pengujian dengan menekan tombol menggunakan kecepatan yang berbeda, baik secara perlahan maupun cepat.

11. Mengubah nilai `DEBOUNCE_DELAY` menjadi 10 ms, kemudian melakukan Build dan Upload kembali. Setelah itu, menekan tombol dan mengamati perubahan kedua penghitung.

12. Mengubah nilai `DEBOUNCE_DELAY` menjadi 200 ms, kemudian melakukan Build dan Upload kembali.

13. Menekan tombol secara berulang dengan cepat dan mengamati perbedaan hasil pembacaan dibandingkan dengan penggunaan waktu debounce sebelumnya.
14. Membandingkan hasil pengujian dengan waktu debounce 10 ms, 50 ms, dan 200 ms untuk mengetahui pengaruh durasi penyaringan terhadap kestabilan pembacaan dan respons tombol.

1.5 Percobaan 5: Akses GPIO Analog dengan Pembacaan ADC Menggunakan LDR

1. Menyiapkan board ESP32 DevKit, sensor LDR, resistor 10kΩ, breadboard, kabel jumper, dan kabel USB.
2. Memastikan konfigurasi `platformio.ini` menggunakan platform `espressif32`, board `esp32dev`, dan framework Arduino seperti pada percobaan sebelumnya.
3. Memasang LDR pada breadboard sebagai bagian dari rangkaian pembagi tegangan.
4. Menghubungkan salah satu kaki LDR ke sumber tegangan 3.3V, sedangkan kaki lainnya dihubungkan ke GPIO 34 dan resistor 10kΩ.
5. Menghubungkan ujung resistor 10kΩ lainnya ke GND sehingga titik pertemuan antara LDR dan resistor menjadi masukan analog pada GPIO 34.
6. Membuka file `src/main.cpp`, kemudian menuliskan program pembacaan ADC menggunakan fungsi `analogRead()` pada pin GPIO 34.
7. Menambahkan perhitungan konversi nilai ADC menjadi tegangan menggunakan rumus nilai ADC dikalikan 3.3 dan dibagi 4095.
8. Melakukan Build untuk memeriksa program, kemudian melakukan Upload untuk mengunggah program ke ESP32.
9. Membuka Serial Monitor dengan baud rate 115200 untuk melihat nilai ADC mentah dan hasil konversi tegangan yang ditampilkan setiap 300 ms.
10. Menutup permukaan sensor LDR menggunakan tangan untuk mengurangi cahaya yang diterima sensor, kemudian mengamati perubahan nilai ADC dan tegangan.
11. Memberikan cahaya pada sensor menggunakan lampu atau senter, kemudian mengamati kembali perubahan nilai pembacaan pada Serial Monitor.
12. Membandingkan nilai ADC yang diperoleh pada kondisi minim cahaya dan kondisi terang untuk mengetahui pengaruh intensitas cahaya terhadap perubahan resistansi LDR dan tegangan pada rangkaian.

2 Analisis Hasil Percobaan

Berdasarkan praktikum Modul 1 yang telah dilaksanakan, diperoleh pemahaman mengenai penggunaan mikrokontroler STM32 dan ESP32 dalam menjalankan berbagai fungsi input dan output. Pengujian mencakup pemrograman LED, penggunaan framework yang berbeda, konfigurasi pull-up dan pull-down, penerapan debouncing, serta pembacaan sinyal analog menggunakan sensor LDR. Adapun analisis dari setiap percobaan dijelaskan sebagai berikut.

2.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

Pada percobaan pertama, dilakukan pengujian terhadap board STM32 Blackpill dengan menjalankan program Blink menggunakan framework Arduino. Program tersebut bertujuan untuk mengendalikan LED bawaan yang terhubung pada pin PC13 agar dapat menyala dan mati secara bergantian dengan durasi masing-masing 500 ms. Berdasarkan karakteristik LED pada board STM32 Blackpill, LED bekerja dengan sistem aktif LOW. Artinya, LED akan menyala ketika pin berada pada kondisi LOW dan akan mati ketika berada pada kondisi HIGH.

Melalui percobaan ini, dapat diketahui bahwa pin GPIO pada mikrokontroler dapat difungsikan sebagai keluaran digital untuk mengontrol perangkat elektronik. Pengaturan tersebut dilakukan melalui fungsi `pinMode()` untuk menentukan mode pin dan `digitalWrite()` untuk mengubah kondisi keluarannya. Penggunaan framework Arduino juga membuat proses pemrograman menjadi lebih sederhana karena fungsi-fungsi pengendalian perangkat keras telah disediakan dalam bentuk yang mudah digunakan.

2.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

Pada percobaan kedua, dilakukan implementasi program Blink pada board ESP32 dengan memanfaatkan framework ESP-IDF. Program digunakan untuk mengatur LED onboard yang terhubung pada GPIO 2 sehingga dapat berubah dari kondisi menyala ke mati secara berulang dengan interval 500 ms. Berbeda dengan framework Arduino, ESP-IDF menggunakan fungsi `app_main()` sebagai awal eksekusi program, bukan menggunakan pasangan fungsi `setup()` dan `loop()`.

Pengaturan kondisi GPIO pada framework ESP-IDF dilakukan menggunakan fungsi `gpio_set_direction()` untuk menentukan arah pin dan `gpio_set_level()` untuk mengatur nilai logikanya. Sementara itu, fungsi `vTaskDelay()` digunakan untuk memberikan jeda waktu pada program. Dari pengujian ini terlihat bahwa ESP-IDF memiliki struktur pemrograman yang lebih berkaitan langsung dengan sistem internal ESP32 dan FreeRTOS. Di sisi lain, Arduino menyediakan pendekatan pemrograman yang lebih praktis sehingga lebih mudah digunakan untuk percobaan dasar.

2.3 Percobaan 3: Pull-up dan Pull-down Eksternal vs Internal

Pada percobaan ketiga, dilakukan pengamatan terhadap penggunaan resistor pull-up dan pull-down melalui dua metode, yaitu pemasangan resistor secara eksternal dan pemanfaatan resistor internal pada ESP32. Kedua konfigurasi tersebut digunakan untuk memastikan bahwa pin GPIO memiliki kondisi logika yang jelas ketika pushbutton tidak sedang ditekan. Hal ini diperlukan agar pin tidak mengalami kondisi floating yang dapat menyebabkan hasil pembacaan menjadi tidak menentu.

Dalam rangkaian pull-up, kondisi awal pin GPIO berada pada logika HIGH karena terhubung dengan sumber tegangan melalui resistor. Ketika tombol ditekan, pin akan terhubung ke GND sehingga berubah menjadi LOW. Sebaliknya, pada rangkaian pull-down, pin berada pada kondisi LOW ketika tombol tidak ditekan dan berubah menjadi HIGH saat tombol ditekan.

Hasil pengujian menunjukkan bahwa penggunaan resistor eksternal maupun internal memiliki tujuan yang sama, yaitu menjaga kestabilan kondisi logika pada input GPIO. Perbedaan utama kedua metode terletak pada cara penerapannya. Resistor eksternal memerlukan komponen tambahan yang dipasang pada breadboard, sedangkan resistor internal dapat diaktifkan langsung melalui pengaturan program menggunakan `INPUT_PULLUP` dan `INPUT_PULLDOWN`. Dengan demikian, penggunaan resistor

internal dapat mengurangi kebutuhan komponen tambahan pada rangkaian.

2.4 Percobaan 4: Debouncing pada Input Tombol GPIO

Pada percobaan keempat, dilakukan pengujian terhadap pengaruh fenomena bouncing pada pushbutton serta penerapan metode debouncing untuk mengatasinya. Bouncing merupakan kondisi ketika kontak mekanis di dalam tombol mengalami getaran sesaat pada saat ditekan atau dilepaskan. Akibatnya, sinyal yang diterima mikrokontroler dapat berubah beberapa kali dalam waktu singkat meskipun pengguna hanya melakukan satu kali penekanan tombol.

Untuk mengurangi kesalahan pembacaan tersebut, digunakan metode debouncing dengan membandingkan nilai penghitung tanpa penyaringan dan penghitung yang telah menerapkan penyaringan sinyal. Pengujian dilakukan menggunakan variasi waktu `DEBOUNCE_DELAY`, yaitu 10 ms, 50 ms, dan 200 ms. Perbedaan nilai waktu tersebut digunakan untuk mengetahui pengaruh durasi penundaan terhadap kestabilan pembacaan tombol.

Penggunaan waktu debounce yang lebih panjang memberikan waktu tambahan bagi sinyal untuk mencapai kondisi stabil sebelum diterima sebagai input. Namun, apabila durasinya terlalu besar, sistem dapat mengalami keterlambatan dalam merespons penekanan tombol berikutnya. Sebaliknya, waktu debounce yang terlalu singkat berpotensi belum mampu menghilangkan seluruh efek bouncing. Oleh karena itu, pemilihan durasi debounce perlu mempertimbangkan keseimbangan antara keakuratan pembacaan dan kecepatan respons sistem.

2.5 Percobaan 5: Akses GPIO Analog dengan Pembacaan ADC Menggunakan LDR

Pada percobaan kelima, dilakukan pengujian pembacaan sinyal analog menggunakan sensor Light Dependent Resistor (LDR) yang dihubungkan ke GPIO 34 pada ESP32. Sensor LDR dimanfaatkan untuk mendeteksi perubahan intensitas cahaya melalui perubahan nilai resistansinya. Dalam rangkaian ini, LDR dipasangkan dengan resistor 10kΩ untuk membentuk rangkaian pembagi tegangan sehingga perubahan resistansi sensor dapat menghasilkan perubahan tegangan pada pin masukan analog.

Pembacaan tegangan dilakukan menggunakan fitur Analog to Digital Converter (ADC) yang berfungsi mengubah sinyal analog menjadi nilai digital agar dapat diproses oleh mikrokontroler. ESP32 menggunakan resolusi ADC sebesar 12-bit, sehingga rentang nilai pembacaan secara teoritis berada pada angka 0 hingga 4095. Nilai tersebut kemudian dikonversikan menjadi perkiraan tegangan dengan menggunakan perbandingan terhadap tegangan referensi sebesar 3.3V.

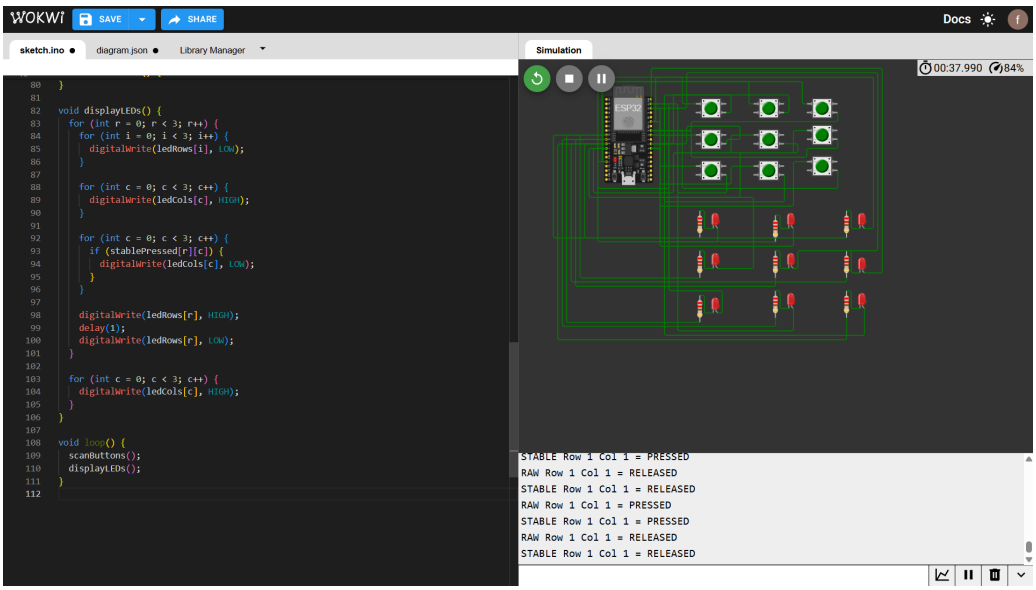
Ketika permukaan LDR ditutupi, intensitas cahaya yang diterima sensor berkurang sehingga resistansinya mengalami perubahan. Sebaliknya, ketika sensor diberikan paparan cahaya dari lampu atau senter, resistansinya akan berubah sesuai dengan peningkatan intensitas cahaya. Perubahan tersebut memengaruhi tegangan pada rangkaian pembagi tegangan dan menghasilkan nilai ADC yang berbeda. Melalui percobaan ini, dapat dipahami bahwa fitur ADC pada ESP32 memungkinkan mikrokontroler membaca perubahan kondisi lingkungan melalui sensor analog, bukan hanya mendeteksi dua keadaan seperti pada input digital.

3 Hasil Tugas Modul

3.1 Hasil Simulasi

Berdasarkan simulasi yang telah dilakukan menggunakan Wokwi, rangkaian Matrix Pushbutton 3x3 berhasil dirancang menggunakan ESP32. Rangkaian terdiri dari sembilan pushbutton dan sembilan LED yang disusun dalam bentuk matriks 3x3. Setiap tombol dirancang untuk mengendalikan LED pada posisi yang sama. Dengan demikian, ketika tombol pada baris dan kolom tertentu ditekan, LED pada baris dan kolom yang bersesuaian akan menyala.

Program menggunakan konfigurasi GPIO sebagai input dan output untuk melakukan pembacaan tombol serta mengendalikan LED. Selain itu, diterapkan metode debouncing dengan waktu tunda sebesar 50 ms untuk memastikan perubahan kondisi tombol telah stabil sebelum diproses oleh mikrokontroler. Pengujian juga dilakukan menggunakan Serial Monitor untuk mengamati perubahan kondisi tombol, baik saat ditekan maupun dilepaskan.



Gambar 1: Rangkaian Simulasi Matrix Pushbutton 3x3 dan Matrix LED 3x3

3.2 Analisis Hasil Tugas Modul

3.2.1 Analisis Kebutuhan Debouncing pada Matrix Pushbutton

Debouncing tetap diperlukan pada rangkaian Matrix Pushbutton 3x3 meskipun pada umumnya hanya satu tombol yang ditekan dalam satu waktu. Hal ini disebabkan oleh karakteristik mekanis pushbutton yang dapat mengalami bouncing ketika ditekan atau dilepaskan. Bouncing merupakan kondisi ketika kontak logam di dalam tombol mengalami getaran atau pantulan kecil sehingga menghasilkan beberapa perubahan sinyal HIGH dan LOW dalam waktu yang sangat singkat sebelum mencapai kondisi stabil.

Kondisi tersebut dapat menyebabkan mikrokontroler membaca satu kali penekanan tombol sebagai beberapa kali penekanan. Oleh karena itu, debouncing digunakan untuk menyaring perubahan sinyal yang tidak stabil tersebut. Pada program yang dibuat, metode debouncing diterapkan dengan waktu sebesar 50 ms. Perubahan kondisi tombol hanya akan diterima apabila sinyal telah berada dalam kondisi yang sama selama waktu tersebut. Dengan demikian, pembacaan tombol menjadi lebih

stabil dan dapat mengurangi kesalahan dalam pengendalian LED.

3.2.2 Analisis Konfigurasi Pull-up pada Pembacaan Tombol

Pada rangkaian Matrix Pushbutton 3x3, konfigurasi yang digunakan untuk membaca kolom tombol adalah internal pull-up dengan menggunakan fungsi `INPUT_PULLUP`. Konfigurasi ini membuat pin GPIO memiliki hubungan internal menuju tegangan HIGH melalui resistor pull-up. Ketika tombol tidak ditekan, pin input akan terbaca dalam kondisi HIGH karena terhubung ke tegangan 3,3 V melalui resistor internal.

Sebaliknya, ketika tombol ditekan pada baris yang sedang diaktifkan, pin kolom akan terhubung ke jalur LOW sehingga menghasilkan pembacaan LOW. Dengan konfigurasi tersebut, kondisi tombol dapat dibedakan berdasarkan nilai logika yang diterima oleh ESP32.

Kondisi pin tidak boleh dibiarkan floating karena pin input yang tidak memiliki referensi tegangan tetap dapat menangkap gangguan listrik dari lingkungan sekitarnya. Hal ini menyebabkan nilai pembacaan menjadi tidak menentu, terkadang HIGH dan terkadang LOW meskipun tombol tidak ditekan. Penggunaan pull-up berfungsi memberikan kondisi logika default yang stabil sehingga pembacaan tombol menjadi lebih akurat.

3.2.3 Analisis Perbedaan Digital Input dan Analog Input

Pembacaan digital input dan analog input memiliki perbedaan mendasar pada jenis sinyal yang diterima oleh mikrokontroler. Digital input digunakan untuk membaca dua kondisi logika, yaitu HIGH dan LOW. Pada rangkaian pushbutton, kondisi tersebut digunakan untuk mengetahui apakah tombol sedang ditekan atau tidak. Oleh karena itu, hasil pembacaan digital hanya menunjukkan dua kemungkinan kondisi.

Sementara itu, analog input menggunakan Analog-to-Digital Converter (ADC) untuk mengubah tegangan analog menjadi nilai digital. Berbeda dengan digital input, ADC mampu merepresentasikan berbagai tingkat tegangan dalam rentang tertentu. Sebagai contoh, ESP32 memiliki ADC dengan resolusi hingga 12-bit yang dapat menghasilkan nilai pembacaan dari 0 sampai 4095. Hal ini memungkinkan mikrokontroler membedakan lebih dari dua kondisi berdasarkan besar tegangan yang diterima.

Apabila salah satu tombol pada matrix digantikan dengan LDR, perubahan intensitas cahaya akan menyebabkan perubahan nilai resistansi LDR dan tegangan pada rangkaian pembagi tegangan. Tegangan tersebut kemudian dibaca melalui ADC sehingga menghasilkan nilai yang bervariasi sesuai dengan intensitas cahaya yang diterima. Dengan demikian, penggunaan ADC memungkinkan sistem tidak hanya mengenali kondisi aktif atau tidak aktif, tetapi juga mengukur perubahan suatu besaran secara bertingkat.

4 Kesimpulan

Berdasarkan praktikum Modul 1 yang telah dilaksanakan, dapat disimpulkan bahwa PlatformIO pada Visual Studio Code dapat digunakan untuk memprogram mikrokontroler STM32 dan ESP32 dengan framework Arduino maupun ESP-IDF. Praktikum ini memberikan pemahaman mengenai penggunaan GPIO sebagai digital output untuk mengendalikan LED dan digital input untuk membaca kondisi pushbutton.

Selain itu, konfigurasi pull-up dan pull-down diperlukan untuk menjaga kestabilan pembacaan input agar tidak mengalami kondisi floating. Penerapan debouncing juga membantu mengurangi kesalahan pembacaan akibat bouncing pada tombol mekanis. Penggunaan ADC pada ESP32 memungkinkan pembacaan sinyal analog dari sensor LDR berdasarkan perubahan intensitas cahaya.

Melalui praktikum ini, praktikan memperoleh pemahaman dasar mengenai pemrograman mikrokontroler, pengendalian perangkat elektronik, serta penerapan input dan output dalam sistem tertanam.

5 Lampiran

5.1 Dokumentasi saat praktikum

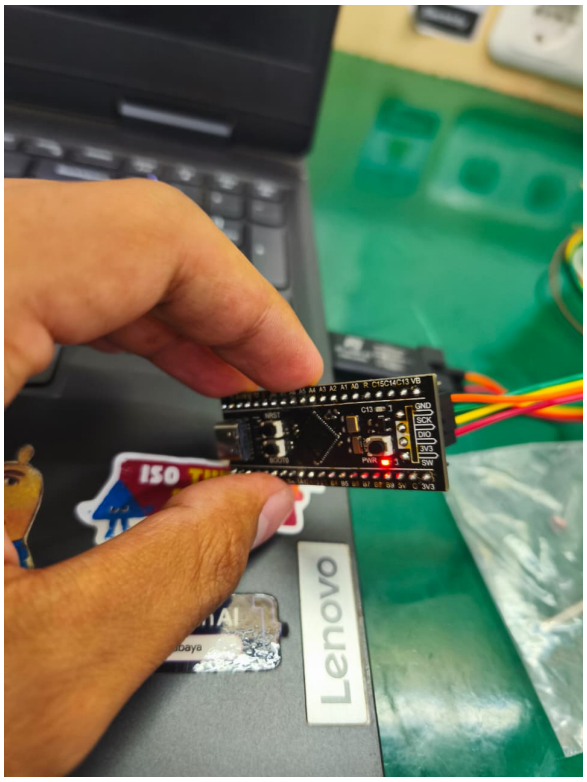
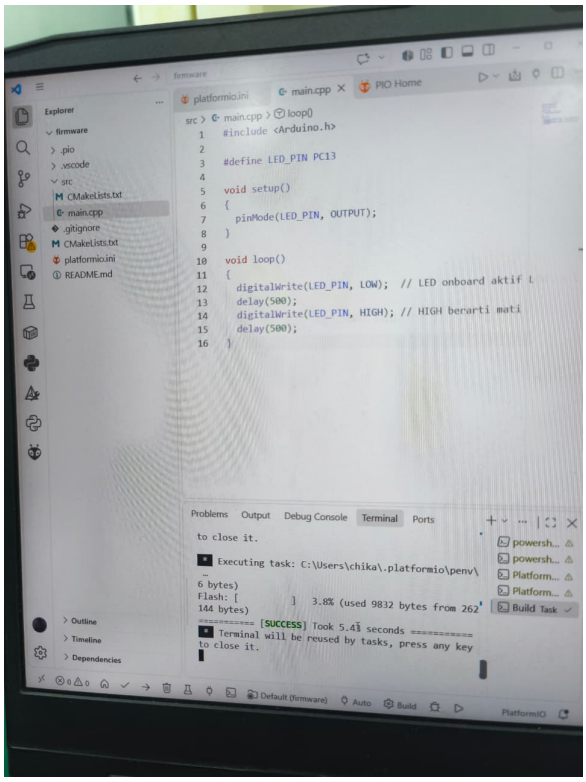


Foto Percobaan 1



Kode Percobaan 1

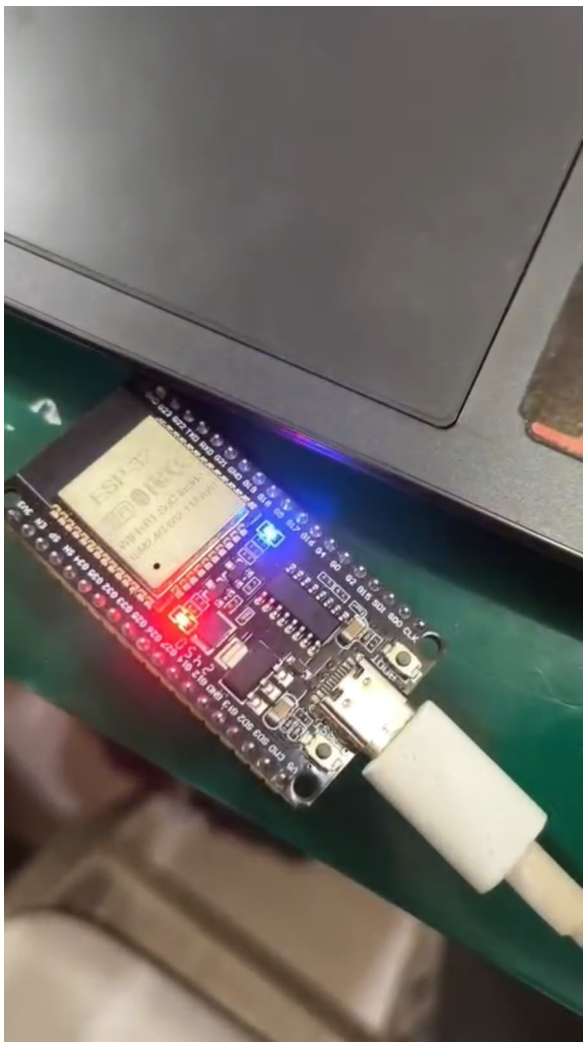
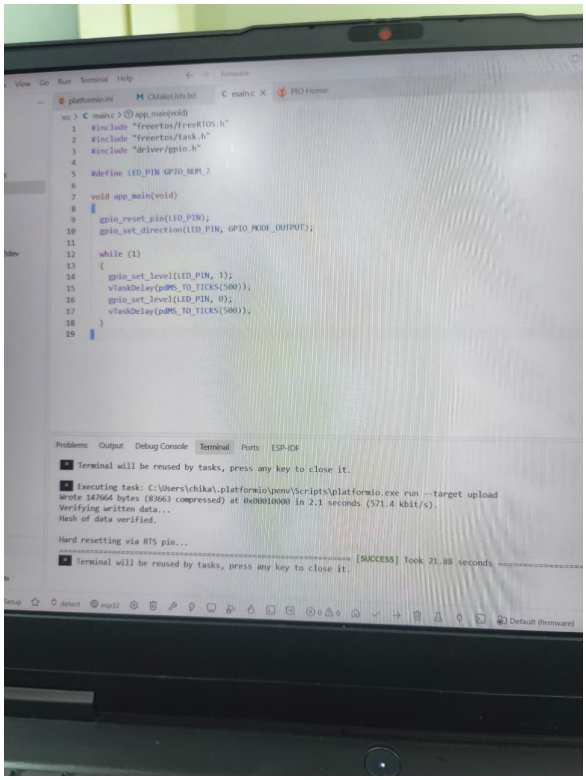


Foto Percobaan 2



```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"

#define LED_PIN GPIO_NUM_2

void app_main(void)
{
  gpio_reset_pin(LED_PIN);
  gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);

  while (1)
  {
    gpio_set_level(LED_PIN, 1);
    vTaskDelay(pdMS_TO_TICKS(500));
    gpio_set_level(LED_PIN, 0);
    vTaskDelay(pdMS_TO_TICKS(500));
  }
}
```

```
Terminal will be reused by tasks, press any key to close it.

Executing task: C:\Users\chika\.platformio\penv\Scripts\platformio.exe run --target upload
Wrote 147664 bytes (83663 compressed) at 0x00010000 in 2.1 seconds (571.4 kbit/s).
Verifying written data...
Hash of data verified.

Hard resetting via RTS pin...

Terminal will be reused by tasks, press any key to close it.
```

Kode Percobaan 2

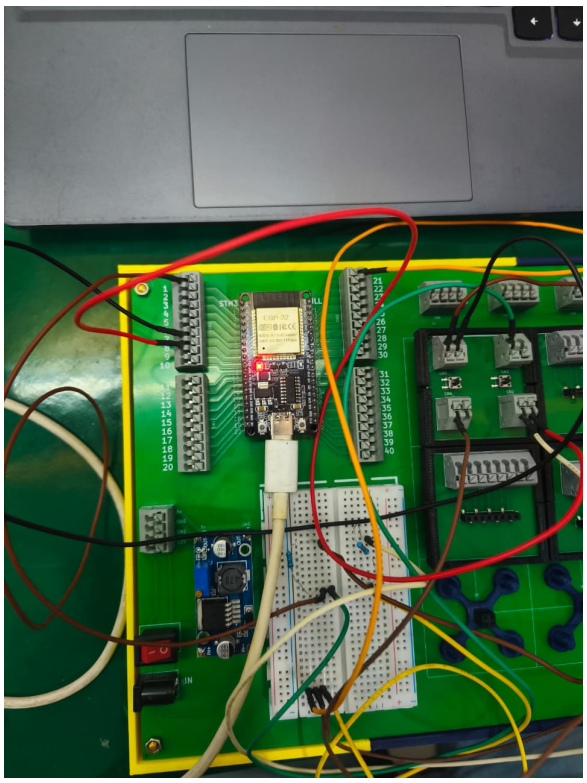
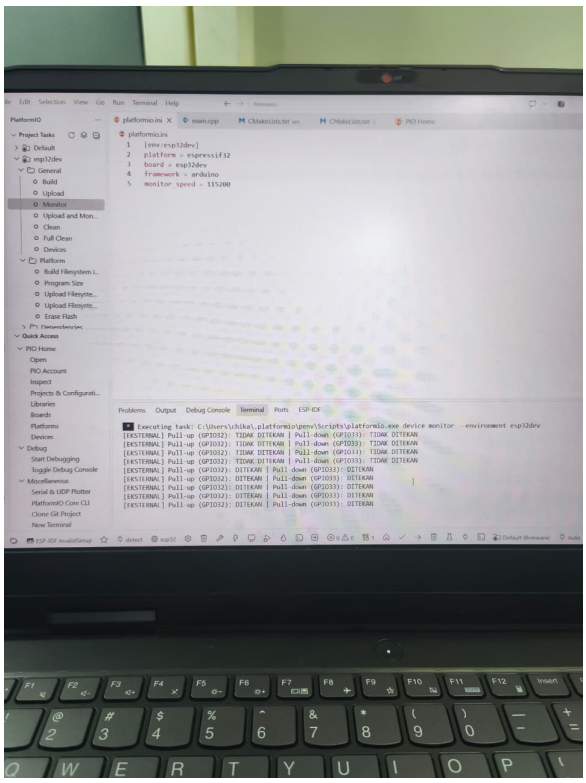
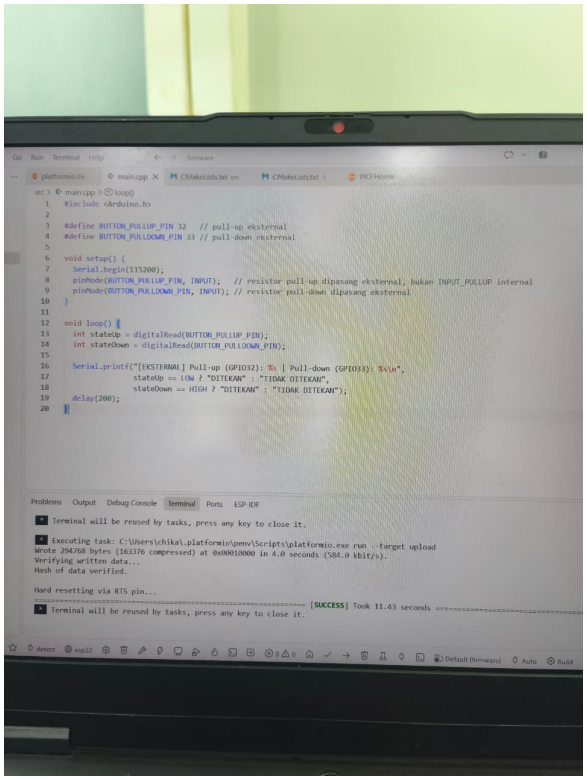


Foto Rangkaian Percobaan 3



Serial Monitor Percobaan 3



```
#include <Arduino.h>

#define BUTTON_PULLUP_PIN 32   // pull-up eksternal
#define BUTTON_PULLDOWN_PIN 33 // pull-down eksternal

void setup() {
  Serial.begin(115200);
  pinMode(BUTTON_PULLUP_PIN, INPUT);   // resistor pull-up dipasang eksternal, bukan INPUT_PULLUP internal
  pinMode(BUTTON_PULLDOWN_PIN, INPUT); // resistor pull-down dipasang eksternal
}

void loop() {
  int stateUp = digitalRead(BUTTON_PULLUP_PIN);
  int stateDown = digitalRead(BUTTON_PULLDOWN_PIN);

  Serial.printf("[EKSTERNAL] Pull-up (GPIO32): %s | Pull-down (GPIO33): %s\n",
                stateUp == LOW ? "DITEKAN" : "TIDAK DITEKAN",
                stateDown == HIGH ? "DITEKAN" : "TIDAK DITEKAN");
  delay(200);
}
```

Kode Percobaan 3

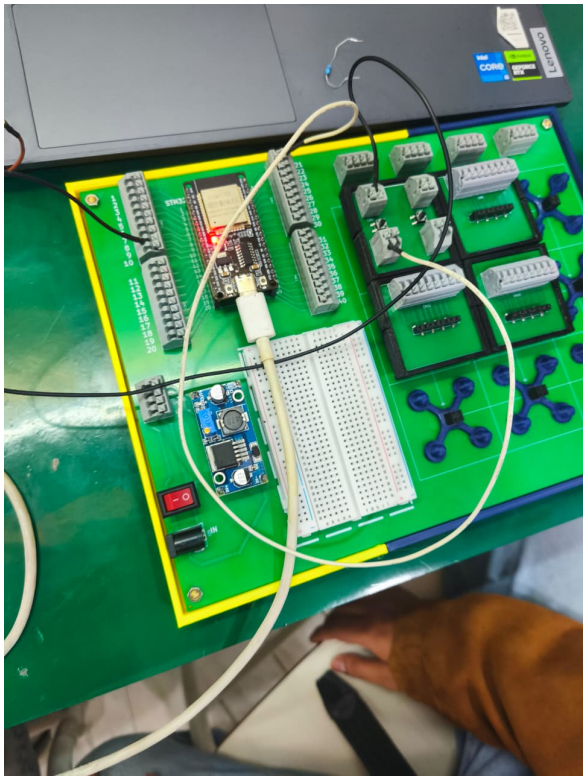
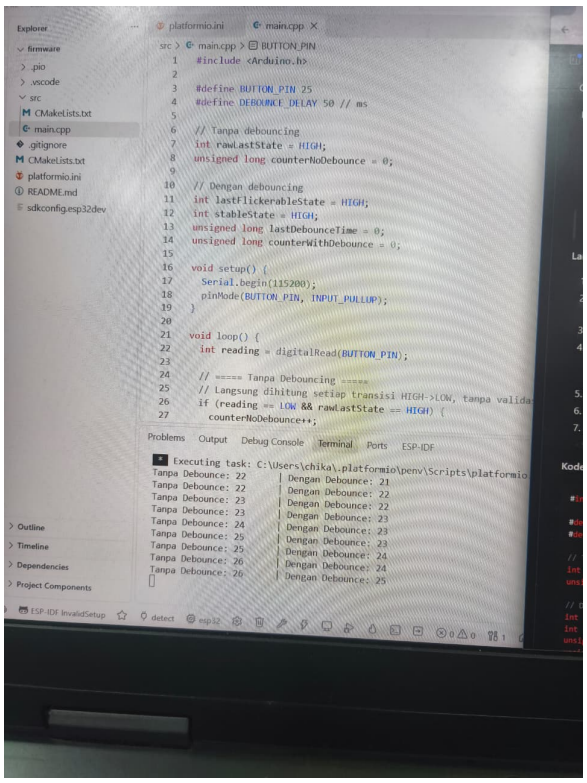


Foto Rangkaian Percobaan 4



Debounce Percobaan 4

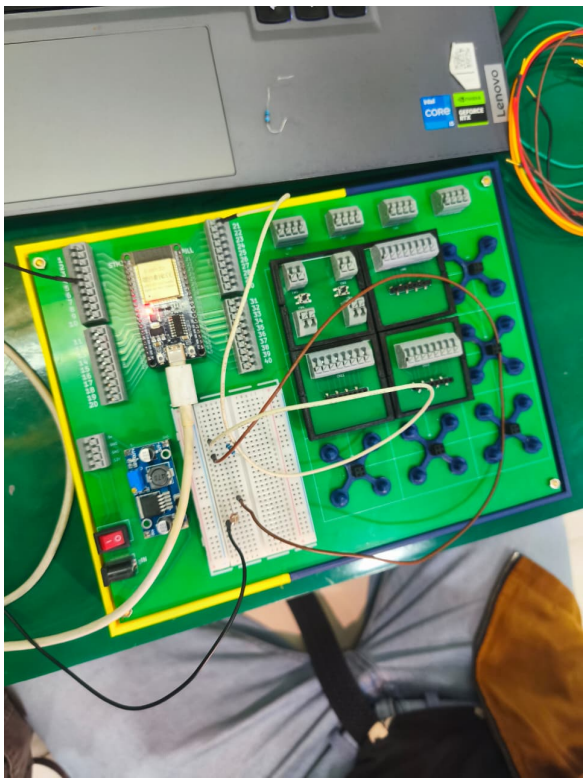
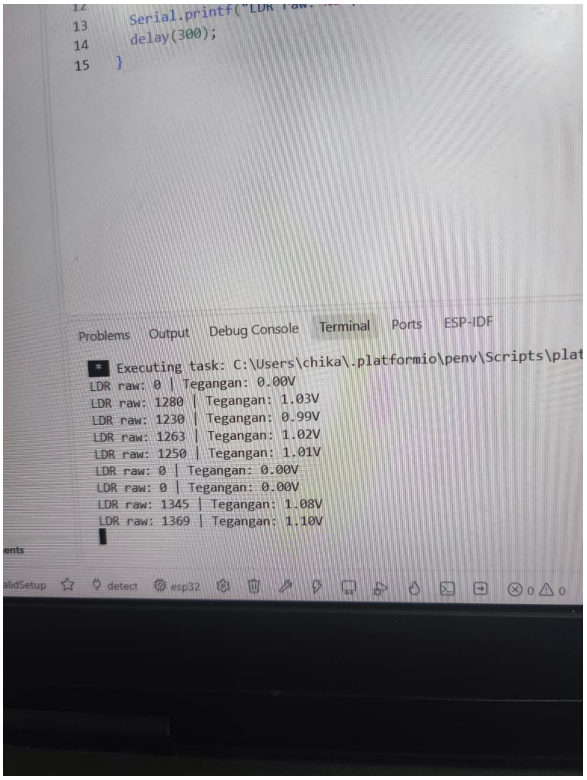


Foto Rangkaian Percobaan 5



Hasil Pembacaan Sensor LDR